Software Requirements Specification

EduMatch AI

Full-stack, AI-assisted school placement
Hackathon demonstration release - v1.0

Document control	Value
Team / challenge owner	EduMatch AI - Group 3 / NESA
Prepared / target demonstration	30 September 2026 / 1 October 2026 (tomorrow)
Audience	Frontend, backend and AI developers; team lead; judges
Status	Implementation specification for a demo; institutional policy validation pending

The demo to build

A NESA officer opens a web dashboard, loads a synthetic cohort and school capacities, runs placement proposals, inspects reasons and unassigned cases, approves or changes proposals, and exports approved results. A locally trained model ranks eligible alternatives when a student has opted into fallback recommendations. Every placement passes deterministic eligibility, capacity and uniqueness checks.

Release boundary
This document specifies the software to build; it is not the application source code. The requested deliverable is an SRS PDF. Demonstration data, thresholds and priority rules are explicitly synthetic and are not presented as NESA-approved policy.

Document map

Pages	Contents
2-3	Scope, assumptions, users, architecture and required screens
4-5	Functional requirements and human approval workflow
6-9	Data contracts, embedded AI, allocation logic and API contracts
10-11	Quality requirements, measurable acceptance tests and test fixture
12-13	Build schedule, judge demonstration, open questions and sources

Scope, users and assumptions

1. Purpose and source of the problem

The supplied NESA challenge card describes matching O-Level students using examination results, capacity and preferences as a manual, time-intensive process. It requests an AI-assisted recommendation engine with human approval and a pilot covering one historical cohort in one province. Treat these as challenge statements, not independently measured operational findings [S1]. The team canvas identifies NESA workflow, bottlenecks and baseline metrics as still awaiting confirmation [S2].

2. Demo scope and priorities

Priority	Required scope
P0 - must ship	Officer login; seed/load CSV data; validation report; school-offer capacity list; run proposals; AI fallback ranking; explanations; exception queue; approve/change/reject; CSV export; reset; model status.
P1 - only after P0 passes	Dashboard filters; bulk approval of unchanged valid proposals; optional capacity what-if comparison with a fresh run.
Later release	NESA production integration; real historical training; student/parent portal; SMS; appeals workflow; national-scale optimization; verified policy implementation.

Target one synthetic province and one cohort: 200 students, 12 school offers and three ordered choices per student. Also support a 500-student stress fixture. These are build targets, not claims about national cohort sizes. A school offer means a school plus admission stage/programme and capacity for that cohort.

3. Users and authority

Role	Permissions and responsibility
Officer	View inputs, generate draft proposals, inspect AI scores, approve or change placements with reasons, export approvals.
Demo administrator	Seed/reset synthetic data, import files, configure demo rules and inspect model metadata. May also act as an officer.
Students / schools	Affected stakeholders represented in data; no account or live notification is required tomorrow.

4. Explicit assumptions

A1. Admission stage is unconfirmed. Use generic offer IDs until NESA confirms entry into O-Level versus placement after O-Level. **A2.** Demo scores use 0-100 with higher values better; convert actual grading only with an agreed mapping. **A3.** Use ranked choices and per-offer capacity. **A4.** Historical individual records may be unavailable; use synthetic records. **A5.** Fallback opt-in is a demo preference field. **A6.** No direct NESA API is required.

Architecture and user interface

5. Recommended implementation

Layer	Selected approach	Responsibilities
Frontend	React + Vite; plain CSS or the team's existing UI library	Five officer screens; API calls; validation feedback; review actions; downloadable CSV.
Backend	Python + FastAPI [S4]	Authentication, CSV validation, model inference, placement orchestration, atomic approval and export.
Database	SQLite for local demo; SQLAlchemy models	Cohort inputs, rules, runs, proposals, final approvals and audit events. Persistent across restarts.
AI component	scikit-learn RandomForestRegressor [S5]; packaged model artifact	Load once at startup; score feasible fallback pairs inside the backend process; no paid API or GPU.
Runtime	One laptop; local frontend and backend	Run without venue internet after dependencies are installed. Hosted preview is optional, not a release dependency.

Request flow
Browser -> FastAPI -> validated cohort snapshot -> model inference + allocation rules -> SQLite draft run -> officer review -> atomic approval -> approved CSV. The browser never performs authoritative capacity calculations.

6. Required screens

Screen	Visible controls and information
Login	Seeded demo officer; sign-in state; logout. Keep credential configuration outside source control.
Dashboard / data	Cohort counts; data source badge; import/seed action; rejected rows; current rules; model loaded/version; run action.
Schools and offers	School, stage/programme, capacity, eligibility threshold, proposed count, approved count and remaining seats.
Placement workspace	Student search; result, choices, proposed offer, rank, status and reason; separate unassigned/invalid queue; student detail panel.
Review and export	Feasible alternatives with AI score; approve/change/reject; mandatory change reason; approved-only CSV; reset button for administrator.

7. Interface behavior

Use English for the first release and clear labels: "Draft proposal", "Approved", "Unassigned", "Synthetic data" and "Demo policy". Display AI compatibility as a score, never admission probability. Show loading/error states; disable duplicate run clicks; warn when data or capacity edits invalidate an existing run. Display measured runtime rather than a fixed claim.

Functional requirements: inputs and AI

All requirements on pages 4-5 are **P0**. "Shall" identifies a release requirement. Tests on page 11 provide the acceptance checks.

ID	Requirement and observable result
FR-01	**Authenticate officers.** The backend shall require an authenticated officer session for cohort, run, review and export endpoints. A seeded synthetic demo account shall work; unauthorized requests shall return 401.
FR-02	**Provide repeatable demo data.** An administrator shall load a 200-student synthetic cohort, 12 offers, ordered choices and demo rules with one action. Reset shall reproduce the same dataset and fixed random seed.
FR-03	**Import CSV inputs.** The system shall accept students.csv, offers.csv and preferences.csv. It shall validate the batch before committing it and return file, row, field and reason for each error. Any error prevents the batch commit.
FR-04	**Validate records.** The backend shall reject duplicate IDs, unknown offers, missing/out-of-range scores, invalid capacities, duplicate preference ranks and repeated choices. Blank scores shall not silently become zero.
FR-05	**Manage rules and capacities.** The administrator shall view and edit demo thresholds, offer capacities and fallback setting. Edits shall version inputs and mark current draft runs stale; approved results require explicit reset before changing the cohort.
FR-06	**Generate draft proposals.** The officer shall start a run on a validated snapshot. The backend shall propose no more than one offer per student, respect available seats and include every data-valid applicant in the result, including unassigned cases.
FR-07	**Run embedded AI inference.** The backend shall load the packaged local model, return ranked feasible fallback offers and record model version and score. It shall exclude ineligible/full offers before proposing a fallback.
FR-08	**Preserve explicit preferences.** The engine shall consider listed choices in rank order under the demo priority policy. It shall propose an unlisted AI-ranked offer only for students whose allow_fallback field is true.
FR-09	**Explain decisions.** Each proposal shall show the applied priority, considered choices, relevant eligibility/capacity reasons, whether fallback was used and the applicable rule/model versions. Explanations shall be generated from recorded decision facts.

Failure behavior
If the model cannot load, display "AI unavailable". Listed-choice proposals may still run in explicitly labeled rules-only mode, but do not claim the AI acceptance gate passed. Never silently substitute random scores or an external service.

Functional requirements: review and export

ID	Requirement and observable result
FR-10	**Show review and exceptions.** Officers shall filter by status, offer and reason; inspect each student's results/choices; and see unassigned reasons such as NO_ELIGIBLE_OFFER or CHOICES_EXHAUSTED.
FR-11	**Require human approval.** Runs shall create drafts only. The backend shall approve a proposal only after rechecking the entire active plan's eligibility, capacity, uniqueness and input versions. No model action shall approve or publish results.
FR-12	**Allow controlled changes.** An officer shall select an alternative and give a non-empty reason. The backend shall replace the student's old reservation atomically and reject changes that breach eligibility, capacity or preference authorization.
FR-13	**Reject a proposal without losing the student.** Rejection shall release its provisional seat, retain an exception record and record the officer's reason. Rejecting one proposal shall not silently reallocate other students.
FR-14	**Prevent stale or duplicate writes.** Approval/change requests shall include the current run/proposal revision. Conflicting or stale requests shall return 409; repeated approval shall not allocate an additional seat.
FR-15	**Export approved decisions.** Export shall include only approved rows with student ID, offer ID, school label, preference rank/fallback flag, officer, decision reason, timestamp and rule/model versions. Drafts shall never appear as final placements.
FR-16	**Record audit history.** The system shall retain actor, action, old/new offer, reason, time and run ID for generate, approve, change, reject and reset actions. Store history in the database, not only in browser state.
FR-17	**Report measured results.** Show valid applicants, assigned, unassigned, approvals, preference satisfaction, violation counts, runtime and current AI mode. Distinguish draft counts from approved counts.
FR-18	**Restore the presentation state.** An administrator shall reset the synthetic demonstration with confirmation. Normal review screens shall not expose a destructive reset action to an officer.

8. Decision states

Run: READY -> GENERATED -> UNDER_REVIEW -> COMPLETED; input edits produce STALE. **Student proposal:** DRAFT -> APPROVED, CHANGED (new draft revision), or REJECTED. A student with no feasible proposal is UNASSIGNED. Rejected and unassigned cases remain visible until an officer resolves them or exports the exception list.

Only one active plan is allowed per cohort. A new run may replace a draft plan, with audit history retained. Once any result is approved, block a fresh allocation run until the administrator explicitly resets the synthetic cohort. This keeps tomorrow's capacity accounting understandable and avoids multiple competing final plans.

Data contracts and storage

9. Required input files

File / grain	Required fields and validation
students.csv One student per row	student_id: unique string; display_name: synthetic alias; cohort_id; province; overall_score: 0-100; math_score, science_score, language_score: 0-100; allow_fallback: true/false. No national ID, phone, address or real name required.
offers.csv One school offer per row	offer_id: unique string; school_name: synthetic label; province; stage_programme: demo label; capacity: non-negative integer; min_overall: 0-100; optional subject_min_*: 0-100; weight_math, weight_science, weight_language: non-negative, sum to 1.
preferences.csv One student-choice pair	student_id; offer_id; rank: 1, 2 or 3. IDs must exist; (student_id, rank) and (student_id, offer_id) must be unique; ranks must be contiguous from 1. Allow 1-3 choices; zero choices are invalid for this demo.
rules.json One versioned configuration	rule_version; score_scale = 0-100; score_direction = higher_better; priority = overall_desc_then_student_id_asc; allow_unlisted_fallback = true; allocation = serial_priority. This is demo policy, never a claimed NESA rule.

Subject fields and profile weights are proposed demo inputs for AI compatibility; they are not claimed to be part of NESA's existing schema. Default eligibility is the offer's overall threshold; subject minima apply only when explicitly configured. Store zero capacity as unavailable, not as missing.

10. Persistent entities and integrity

Entity	Key stored fields / integrity
Cohort + input version	Source label, synthetic flag, active run, rules version, import validation status and checksum.
Student, Offer, Preference	Use the CSV identifiers as keys within the cohort. Foreign keys link preferences to students/offers; reject orphan records.
PlacementRun	run_id, cohort_id, input/rule/model versions, mode, runtime, metrics, status and revision.
Proposal / final decision	Unique (run_id, student_id); nullable offer_id; status, rank, model score, reason codes, trace, revision, reviewer and time. One approved placement per cohort/student.
AuditEvent + ModelMetadata	Before/after decision facts and actor; model file hash, feature schema, training source, seed and evaluation summary.

11. Data acquisition

For tomorrow, generate the files locally; label all records and capacities synthetic. For a later pilot, request authorized, de-identified examination results, ranked choices, school-offer capacity, grading mappings and written allocation rules from the NESA focal person. Request historical decisions and appeal reasons for evaluation, but do not assume historical placements are unbiased training labels. The public NESA site [S3] is an institutional contact source, not a downloadable student-level training dataset.

Embedded AI model specification

12. Model purpose and honest demonstration claim

Implement a local **RandomForestRegressor** to estimate student-offer compatibility from numeric pair features [S5]. It supports the officer by ranking eligible fallback options. It does not set examination priority, eligibility, capacity or final decisions. With no verified outcomes available, train on disclosed synthetic compatibility examples. This demonstrates model integration and inference; it does not establish real-world placement quality.

Item	Required implementation
Pair features	overall_score/100; subject_fit; eligibility_margin; same_province (0 or 1). subject_fit = weighted sum of the three subject scores /100, using offer profile weights. eligibility_margin = max(0, overall - min_overall)/100. Exclude student IDs, names and protected traits.
Synthetic target	y_demo = 0.75 x subject_fit + 0.15 x (overall_score/100) + 0.10 x same_province. This is an invented teaching target, not admission success, acceptance probability or an institutional label.
Training set	Generate 2,000 training students and 12 synthetic offers (up to 24,000 pair rows). Use a fixed seed; keep the visible 200-student demo cohort separate. Save generator settings and source label.
Training procedure	Split pair rows by student ID: 80% training, 20% held-out using GroupShuffleSplit [S6]. Fit preprocessing only on training data. Use about 100 trees, max_depth=8, min_samples_leaf=5 and random_state=42; lock installed dependency versions.
Stored artifact	Save model.joblib plus metadata.json with version, feature order, source=SYNTHETIC, target formula, dependency versions, train/test counts, seed, held-out MAE and baseline MAE.
Runtime output	fit_score = 100 x clipped prediction in [0,1]. Rank eligible fallback offers descending; equal scores use offer_id ascending. Return model version and actual inputs; do not describe score as confidence.

13. Evaluation and explanations

Compare held-out mean absolute error against predicting the training target mean. A release test requires measured MAE below that baseline and identical scores after reloading the artifact; report actual values only. This tests recovery of the synthetic target, not NESA effectiveness. Show academic profile fit and threshold facts as contextual information, not proven causal model explanations. The placement trace explains the allocation rules; a chatbot is unnecessary.

What judges can verify
Show "Local model loaded", model version, synthetic training provenance, held-out metric and a student's fallback ranking. Then change a subject score in a fresh unapproved cohort and recompute the ranking. Never label a hard-coded lookup table as trained AI.

Placement logic and hard constraints

14. Deterministic demo policy

Use a simple serial allocation so the implementation can be completed tomorrow. Process students by overall_score descending, then student_id ascending for equal scores. For each student, test listed choices in rank order and reserve the first eligible offer with a free seat. If no listed choice is feasible and fallback is authorized, rank remaining eligible unlisted offers with the AI model and reserve the best available option. Otherwise retain an unassigned case.

The ID tie-break is only a deterministic demo convention. It is not a fairness policy or a verified NESA priority rule. This algorithm guarantees the stated hard constraints if implemented correctly; it does not guarantee globally optimal preferences, equal outcomes across groups or stability under other allocation policies.

15. Required orchestration

1. Validate inputs and model status; create an immutable input/rule snapshot.
2. Initialize free seats from capacity; approved records cannot coexist with a new run in this demo.
3. Sort all data-valid applicants by the demo priority.
4. Record each considered choice and its eligibility/availability outcome.
5. Reserve one seat for the chosen listed or AI-ranked fallback offer; save the trace.
6. Save null offer and specific reasons when no proposal is feasible.
7. Independently check the full plan; abort on any violation.
8. Persist the run and all proposals in one transaction; then return metrics.

16. Invariants enforced by the backend

Invariant	Check before persisting or approving
Capacity	For each offer, the active plan's DRAFT + APPROVED reservations must not exceed capacity. Approved rows are a subset of these reservations, not extra seats.
Eligibility	Overall and any configured subject minima are met; offer stage/cohort belongs to the current input snapshot.
Uniqueness	At most one non-rejected assigned proposal per student in the active plan and one approved final placement per cohort/student.
Preference authority	Use a listed choice, or an unlisted choice only with allow_fallback=true and the corresponding rule enabled. Officer changes obey the same restriction.
Accountability	Every valid student appears once, assigned or unassigned. Every approval/change has actor, time, version and reason.

17. Review and change transaction

Take a database write transaction; check run and proposal revision; temporarily release the student's old reservation; verify the proposed replacement against all other active reservations; write the replacement and audit event; increment revision; commit. Roll back everything on failure. A full target offer returns 409 CAPACITY_FULL. An ineligible target returns 422 INELIGIBLE_OFFER. Human approval never bypasses these checks.

API and external interface contracts

All routes use the prefix **/api**, JSON responses and authenticated access except login and a minimal health check. Use FastAPI-generated OpenAPI documentation during integration [S4]. UUIDs identify runs/proposals; input IDs remain strings.

Method / route	Contract
POST /auth/login	Input: username, password. Output: session cookie; 401 on failure. Logout clears the session. No credentials in URLs.
POST /demo/seed	Administrator only; deterministic synthetic seed; explicit reset=true required to replace existing data. Returns cohort_id and counts.
POST /cohorts/import	multipart students, offers, preferences and rules. Return cohort_id/counts if valid; 422 with row-level errors and no commit otherwise.
GET /cohorts/{id}/summary	Counts, source/synthetic flag, rules/input version and active_run_id. GET /offers?cohort_id=... returns capacities and reservations.
PATCH /offers/{id}; PATCH /cohorts/{id}/rules	Administrator only; include expected_input_version. Validate edits, increment input version and mark drafts stale; block after any approval. POST /auth/logout clears the session.
GET /model/status	Loaded flag, version, hash, feature schema, training provenance, held-out/baseline MAE and any load error. No model upload endpoint needed.
POST /runs	Input: cohort_id, expected_input_version. Output: run_id, revision, mode, metrics, runtime_ms. Reject overlapping runs with 409.
GET /runs/{id}/proposals	Filters: status, offer_id, student_id. Returns student inputs, proposal, reasons and trace; include unassigned records.
GET /proposals/{id}/alternatives	Returns only eligible alternatives feasible against other reservations, with fit_score/model_version; enforce fallback authorization.
POST /proposals/{id}/decision	Input: action=approve/change/reject, offer_id for change, reason, expected_revision. Output: updated proposal + new revision. Change produces a revised draft requiring approval.
GET /runs/{id}/export.csv	Approved-only CSV; set attachment filename and text/csv. Optional separate exceptions export includes unassigned/rejected cases.

18. Example proposal response


```
{ "student_id": "S003", "offer_id": "O02",
"status": "DRAFT", "preference_rank": 2,
"reason_codes": ["FIRST_CHOICE_FULL", "SECOND_CHOICE_FEASIBLE"],
"fit_score": null, "model_version": "rf-demo-v1",
"rule_version": "demo-policy-v1", "revision": 1 }
```


A null fit_score is valid for a listed-choice proposal: the AI did not determine that assignment. Fallback records include an actual model score and FALLBACK_AI_RANKED. Errors use {code, message, details}; imports add file/row/field. Return 400 for malformed requests, 401/403 for access failures, 409 for stale/capacity conflicts and 422 for validation/eligibility failures. Hide internal stack traces.

Nonfunctional requirements and metrics

ID	Requirement	Acceptance target for demo
NFR-01	Correctness	Zero capacity, eligibility, duplicate-placement or preference-authorization violations in every accepted plan and approval transaction.
NFR-02	Responsiveness	500 valid students / 20 offers: complete a run in <=10 seconds on the presentation laptop after model load. Student detail and review actions target <=2 seconds locally. Record the actual hardware/runtime.
NFR-03	Reliability	Seed, run, approve, export and restart work without data loss. The same inputs, rules, model and seed produce the same proposals; a failed transaction changes no seats.
NFR-04	Offline readiness	After setup, the complete demo works with internet disconnected. Local inference, synthetic data and both applications run on the laptop.
NFR-05	Access and secrets	Server-side role checks; hashed passwords; authenticated sessions; same-site cookie controls and CSRF protection for mutations; local-only services for the laptop demo. Use HTTPS if hosted; no secret committed to Git.
NFR-06	Privacy	Only synthetic student data on the demonstration machine and screenshots. No real student names/identifiers or private records in logs; restrict model artifacts to trusted files packaged by the team.
NFR-07	Usability	Readable at laptop/projector width; keyboard-accessible controls, visible focus and text error messages. Status is represented by text as well as color.
NFR-08	Traceability	100% of proposals have reason codes and input/rule/model provenance; 100% of approvals/changes have an actor and timestamp.
NFR-09	Maintainability	Keep model inference, allocation, validation and API/UI code separate. Include exact dependency lockfiles, setup commands and a README; no request-time retraining.
NFR-10	Transparency	Persistent synthetic-data/demo-policy labels. No unsupported production-accuracy, fairness, efficiency or official-integration claims.

19. Dashboard metric definitions

Placement rate: assigned draft proposals / all data-valid applicants. **Top-three rate:** applicants with a listed rank 1-3 proposal / all data-valid applicants; unassigned and unlisted fallback cases stay in the denominator. **First-choice rate:** rank-1 proposals / all data-valid applicants. **Unassigned rate:** unassigned / all data-valid applicants. **Utilization:** active reservations / total available seats. Use null/"N/A" for a zero denominator.

Display approved counts and approved preference rates separately; do not present draft metrics as final outcomes. Runtime measures backend allocation plus validation, excluding training. Count invalid imported rows separately. Compare a rules-only run with an AI run only on the same fresh input snapshot and report any measured difference without claiming causal real-world benefit.

Acceptance tests and demo fixture

20. Minimum end-to-end acceptance gate

Test / requirement	Pass condition
AT-01 / FR-01-05	Unauthorized requests fail; seed works; bad CSV gives exact row errors and no partial import; zero capacities and tied scores are handled.
AT-02 / FR-06,08-09	The six-student fixture below produces the expected plan, reasons and metrics on repeat runs; all independent invariants pass.
AT-03 / FR-07	Model file loads inside the backend; actual predictions and version appear; held-out synthetic MAE beats mean-target baseline. Artifact reload reproduces predictions.
AT-04 / FR-07-08	Fresh fixture variant: set B capacity=0, C capacity=2 and S3 fallback=true. S3 gets AI-ranked C with a score; S4 still gets C. With S3 fallback=false, S3 remains unassigned.
AT-05 / FR-11-14	Draft export contains no placements. After S1 approval, S1 appears exactly once. Changing S3 from B to full A fails atomically. Stale revision and repeated approval cannot add a seat.
AT-06 / FR-10,13,16	Reject S3: reservation B is released, S3 stays in the exception queue, and actor/reason/history persist after restart.
AT-07 / FR-15,17-18	Approved CSV reconciles with approved database rows; metrics use all valid applicants; synthetic reset restores known counts.
AT-08 / NFR-02,04	Run 500-student fixture within target; rehearse seed -> AI run -> review -> approve -> export with Wi-Fi off.

21. Hand-checkable allocation fixture

Offers: **A** capacity 2 / minimum 50; **B** capacity 1 / minimum 50; **C** capacity 1 / minimum 70. All belong to one stage/province; no subject minima. Subject scores are populated in 0-100. Fallback is false unless explicitly varied in AT-04.

Student	Overall	Ranked choices	Expected draft result
S1	90	A, B	A - first choice
S2	85	A	A - first choice
S3	80	A, B	B - A is full
S4	75	C	C - first choice
S5	65	A, C	Unassigned - A full; C ineligible
S6	45	A	Unassigned - no eligible choice

Expected base metrics: 6 valid applicants; 4 assigned; 2 unassigned; top-three rate 4/6 = 66.7%; first-choice rate 3/6 = 50%; seat utilization 4/4 = 100%; 0 approved before review; 0 hard-constraint violations. Also test tied scores, unknown IDs, duplicate choices, all-zero capacity and a missing model file.

Build plan and judge demonstration

22. Timeboxed implementation before tomorrow

Use elapsed hours from the team's start time. The roles below are workstreams; combine them if fewer people are available. Implement against a shared schema and commit an integrated workflow early.

Time	Owner / work	Completion checkpoint
0-1 h	Team lead + all: confirm scope; agree IDs, CSV/API schema and demo policy	Single agreed data contract; six-student fixture; repository starts locally.
1-4 h	Backend: database/import/allocation. Frontend: login/data/offers screens. AI: generator, training and artifact. Lead: test fixture/pitch facts.	Backend runs base fixture; UI reads real API data; model provenance and metrics saved.
4-7 h	Integrate model fallback, proposal list, reason traces and review transactions	One end-to-end run from UI; fallback uses local model; full/ineligible override blocked.
7-9 h	Complete approval, audit, CSV export, model status and dashboard metrics	P0 functions integrated; approved CSV reconciles; no partial/stale writes.
9-11 h	Run acceptance gate, fix failures, seed 200-student and stress fixtures	Offline test passes; actual performance measured; reset works.
11-12 h	Freeze working version; rehearse pitch; record backup screen capture	Laptop ready; startup instructions checked; pitch and backup capture available.

23. Four-minute demonstration for judges

0:00-0:30: Explain the matching problem and show synthetic-data/demo-policy labels.
0:30-1:15: Show results, ranked choices and capacities; run the cohort and show measured runtime/violations.
1:15-2:10: Inspect a first-choice-full case; show its fallback ranking and actual local model/version.
2:10-3:10: Attempt a full-school change and show rejection; approve a feasible proposal; inspect the audit trail.
3:10-4:00: Export the approved result; show remaining exceptions and explain the next step of NESA rule/data validation.

24. Required handover artifacts from the build team

Working frontend and backend; persistent demo database; seed/CSV fixtures; trained model and metadata; training script; dependency lockfiles; README startup/reset instructions; acceptance results; pitch screenshots and a short backup recording. A hosted link is useful only if already stable. Keep a local functioning copy for the presentation.

Cut scope if time becomes tight
Finish the P0 flow before adding P1 features. Skip chatbots, mobile apps, map views, SMS, live NESA integration and national-scale claims. Do not drop the real model call, independent constraint checks or human approval: those are the core proof.

Validation questions and references

25. Questions to resolve with NESA

Question	Impact on a later validated release
Which placement stage and programmes?	Sets eligible destinations, programme fields and results needed.
What score direction, eligibility, priority and tie-break rules apply?	Replaces invented demo policy; determines the correct allocation algorithm.
How are school capacities and reserved places defined?	Determines seat pools and any authorised special-case constraints.
How many choices, and is unlisted fallback allowed?	Sets preference schema and limits of AI-ranked alternatives.
Which data and interfaces can be shared lawfully?	Determines de-identification, access control, integration and evaluation plan.
What baseline and officer approval workflow are agreed?	Enables measured benefit claims and a usable pilot acceptance process.

These questions do not block a clearly labeled synthetic demo. They do block claims that the prototype implements verified NESA rules or is ready for operational allocation. The source canvas's final statement refers to O-Level graduates and a largely manual process, while its earlier sections flag stage/workflow uncertainty; this SRS preserves those uncertainties [S2].

26. Sources and technical references

[S1] Supplied challenge card. image(1).png, heading "#3 NESA". Source for challenge wording, requested AI/human approval, key inputs and one-cohort/one-province pilot. Team attachment; reviewed 30 September 2026.

[S2] Team problem canvas. EduMatch AI Problem Framing Canvas(1).pptx, six slides, dated 29 September 2026. Source for project name, Group 3, NESA focal person, validation gaps and proposed demo targets. Reviewed 30 September 2026.

[S3] NESA official website. Institutional identity and contact reference only; not a source for the invented rules or student-level data.
https://www.nesa.gov.rw/

[S4] FastAPI official documentation. Backend framework and automatic API documentation reference.
https://fastapi.tiangolo.com/

[S5] scikit-learn: RandomForestRegressor. Official estimator reference; model choice and demo settings are team implementation proposals.
scikit-learn.org / RandomForestRegressor

[S6] scikit-learn: GroupShuffleSplit. Official reference for splitting synthetic pair rows by student group.
scikit-learn.org / GroupShuffleSplit

Public reference pages checked 30 September 2026. Pin the versions installed by the team rather than relying on a moving "stable" documentation version. All thresholds, data volumes, target metrics, stack choices, timelines and formulas in this SRS are proposed demo requirements unless attributed to the supplied project materials.